

PORTFOLIO CASE STUDY

Payment Gateway Integration

for a Global Learning Platform

Dibyashree Chowdhury
Business Analyst | 2.5+ Years Experience

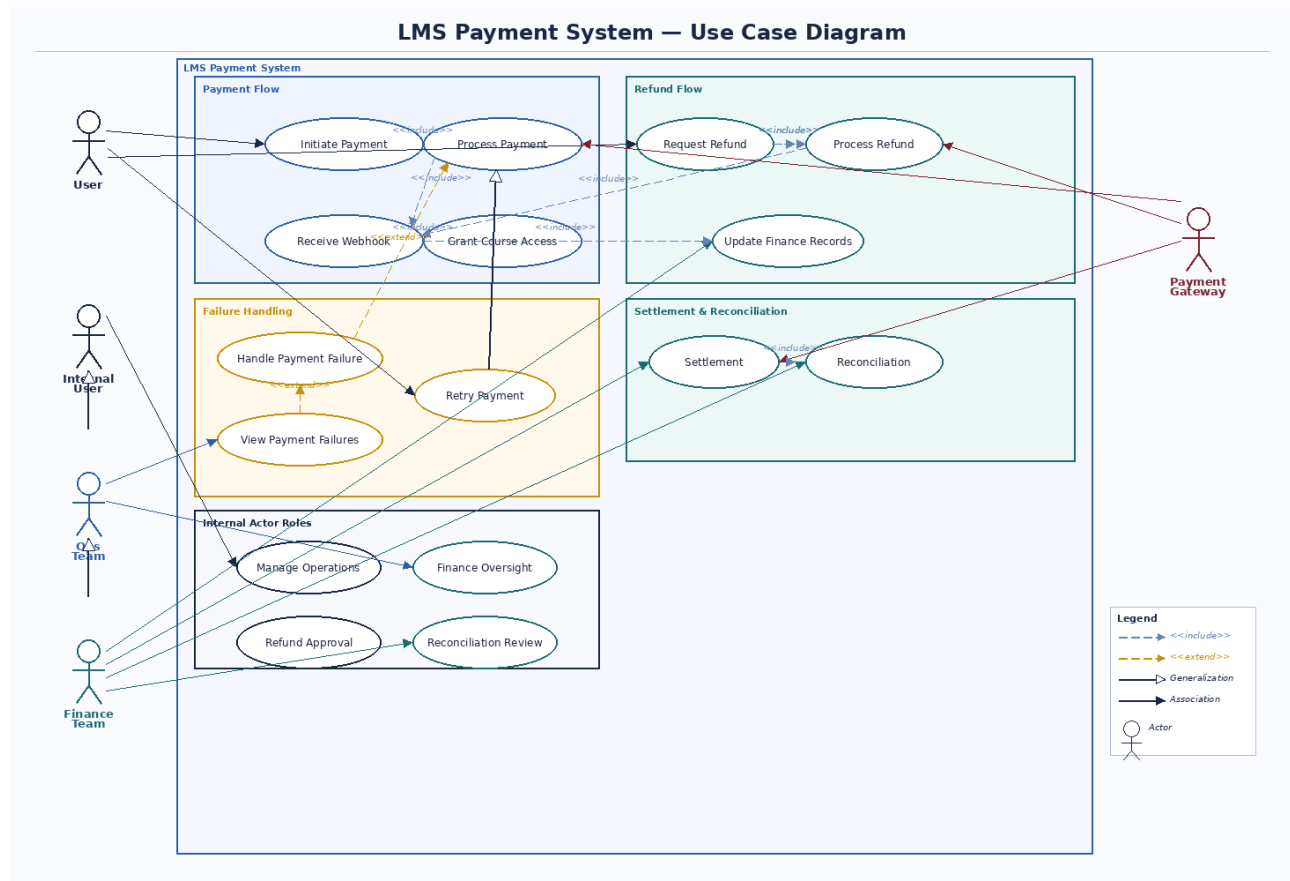
Requirements & FRD	API Testing	UAT & Defects	SQL Analytics	Stakeholder Mgmt
12+ Countries Enabled	20% Faster Checkout	10% Higher Success Rate	3 Payment Gateways	

Healthcare IT • Fintech • Ed-Tech • Automotive Technology

USE CASE DIAGRAM

LMS Payment System — Use Case Diagram

The diagram below shows all actors, use cases, and their relationships (include, extend, and generalization) within the LMS Payment System boundary.



Project Overview

The Business Problem

A fast-growing global learning platform was constrained to domestic-only payments. International users either couldn't complete purchases or faced high failure rates at checkout — directly blocking global revenue expansion.

Checkout abandonment was high, multi-currency support was absent, and the payment system had no retry logic, no refund automation, and no real-time status updates via webhooks.

My Role

As the sole Business Analyst on this project, I owned the feature end-to-end — from stakeholder requirement gathering through FRD documentation, API testing, UAT coordination, and Jira defect management.

Tools & Platforms

Postman	Jira
Razorpay	Stripe
Agile / Scrum	REST APIs

Project Snapshot

- **Industry:** Ed-Tech / Learning Management
- **Domain:** Fintech / Payment Integration
- **Gateways:** Razorpay, Stripe, PayPal
- **Methodology:** Agile (2-week sprints)
- **Duration:** Multi-sprint delivery
- **BA Artefacts:** BRD, FRD, RTM, UAT Plan

Business Goals

- Enable international payment acceptance
- Support multi-currency transactions
- Reduce checkout abandonment rate
- Automate refund and settlement workflows
- Achieve 99.9% system uptime SLA

Stakeholders Engaged

- **Product Team** — Feature scope & priorities
- **Backend Dev Team** — API feasibility
- **Operations** — Refund workflows
- **Finance** — Settlement & reconciliation
- **Customer Support** — UAT & edge cases

Requirement Gathering & Analysis

I structured requirement elicitation across five phases, ensuring every input was validated, documented, and signed off before moving to FRD.

01	Business Alignment Aligned with Product on goals: international expansion, multi-currency, higher success rate, faster checkout.
02	As-Is Flow Analysis Mapped the existing payment flow to identify failure points, drop-offs, and unsupported scenarios.
03	API Feasibility Validation Collaborated with backend devs to validate gateway capability across Razorpay, Stripe, PayPal before documenting requirements.
04	Ops & Finance Discovery Ran dedicated sessions with operations (refund workflows) and finance (settlement timelines, reconciliation).
05	Documentation & Sign-Off Produced BRD and FRD with acceptance criteria, API references, and stakeholder sign-off for each requirement.

Key Functional Requirements Documented (FRD)

FRD ID	Requirement	Priority	Acceptance Criterion
FRD-01	International payment acceptance — multi-currency, multi-gateway	Critical	Successful payment in USD, GBP, EUR with 200 OK response
FRD-02	Failure handling — card declined, unsupported currency, invalid payload	Critical	402/400 returned; friendly error shown; no order created
FRD-03	Retry logic for gateway timeouts and transient network failures	High	Max 3 retries; same order_id reused; no duplicate charge
FRD-04	Idempotency — prevent duplicate transactions on repeated API calls	Critical	409 Conflict returned; existing payment_id reused
FRD-05	Refund workflow automation and webhook-based status updates	High	refund_completed webhook triggers finance update within SLA
FRD-06	Settlement reconciliation between gateway logs and LMS database	High	Zero mismatches in daily reconciliation report

API Testing & Technical Validation

Because this was an API-heavy integration, my BA involvement went beyond documentation. I independently tested all payment APIs in Postman, validating request-response contracts, error handling, retry logic, idempotency, and webhook event flows.

API Tested: POST /api/v1/payments/create

SUCCESS	HTTP 200 OK	Trigger: Valid payload — correct order_id, amount, currency Validated: payment_id returned, status=SUCCESS, course access granted, payment_success webhook triggered
CARD DECLINED	HTTP 402	Trigger: Card declined / insufficient funds Validated: Friendly error displayed, retry option shown, no order created, payment_failed webhook triggered
GATEWAY TIMEOUT	HTTP 408	Trigger: Gateway did not respond within SLA Validated: System retried up to 3 times using same order_id; no duplicate charge created; ops alerted
INVALID PAYLOAD	HTTP 400	Trigger: Missing required fields (e.g., amount) Validated: 400 Bad Request returned; clear validation message; no transaction record created
DUPLICATE	HTTP 409 Conflict	Trigger: Same order_id sent twice (retry / UI bug) Validated: Existing payment_id returned; no new charge created — idempotency confirmed
UNSUPPORTED CURRENCY	HTTP 400	Trigger: Currency not supported by selected gateway Validated: 400 returned with specific error code; user shown supported currency list

Webhook Events Validated

- **payment_success** — Order marked PAID, course access granted, no duplicate
- **payment_failed** — Failure reason logged, no order or access created
- **refund_completed** — Finance records updated, user notified

I also validated duplicate webhook handling — if the same event was delivered twice, the system should not create a duplicate course access or double refund.

Idempotency Validation

Each payment was tied to a unique **order_id**. Repeated requests with the existing payment_id.

Validated in Postman: sent the same request twice — confirmed only

Business consequence of failure: double charge = immediate chargeback

04

SQL & Data Analysis

I used SQL to validate data accuracy, measure business impact, and surface integrity issues post-integration — making data validation a BA discipline, not just a developer task.

Payment Success Rate by Gateway

Queried transaction records segmented by gateway and currency to compare success rates before and after go-live. Identified which gateway performed best for which country-card combination.

```
SELECT gateway, COUNT(*) total,
SUM(CASE WHEN status='SUCCESS' THEN 1
END) success,
ROUND(success*100.0/total, 2)
success_rate FROM payments GROUP BY
gateway
```

Funnel Drop-Off Analysis

Counted user records at each checkout step (course_selected → checkout_initiated → payment_attempted → payment_success → access_granted) to identify the highest drop-off. Found: sharpest fall between payment_attempted and payment_success, pointing to gateway latency and unsupported currency rejections.

```
SELECT step, COUNT(DISTINCT user_id)
users, LAG(COUNT(*)) OVER (ORDER BY
step_order) AS prev_step, 100 -
ROUND(COUNT(*)*100.0/LAG ...) drop_pct
FROM funnel_events GROUP BY step
```

Settlement Reconciliation

Reconciled gateway transaction logs against LMS database records to catch status mismatches — e.g., transactions showing SUCCESS in the gateway but still PENDING in the LMS, indicating a webhook delivery gap.

```
SELECT g.order_id, g.status AS
gateway_status, l.status AS lms_status
FROM gateway_txns g JOIN lms_orders l
ON g.order_id = l.order_id WHERE
g.status != l.status
```

UAT & Defect Management

UAT Scenarios Derived from FRD

Each FRD use case was mapped to a UAT scenario. Positive, negative, and edge-case scenarios were all covered before sign-off.

RTM Mapping

- FRD-01 → US-01 → UAT-PAY-01 → Payment API → **PASS**
- FRD-02 → US-02 → UAT-PAY-02 → Error Handling → **PASS**
- FRD-03 → US-03 → UAT-PAY-03 → Retry Logic → **PASS**
- FRD-04 → US-04 → UAT-PAY-04 → Idempotency → **PASS**
- FRD-05 → US-05 → UAT-PAY-05 → Refund API → **PASS**

UAT ID	Scenario	Expected Result	Status
UAT-PAY-01	Successful international payment via Stripe (USD)	200 OK; access granted; payment_success webhook; PAID status in LMS	PASS
UAT-PAY-02	Card declined — insufficient funds	402 returned; friendly error; retry option shown; no order created	PASS
UAT-PAY-03	Gateway timeout — retry logic validation	Max 3 retries; same order_id; no duplicate charge; 408 on final fail	PASS
UAT-PAY-04	Duplicate payment request with same order_id	409 Conflict; existing payment_id returned; no new charge	PASS
UAT-PAY-05	Refund initiated by ops team	refund_completed webhook received; finance record updated; user notified	PASS
UAT-PAY-06	Webhook delay — payment stuck in PENDING (P0 defect)	Blocked release; manual reconciliation; retry mechanism implemented	FIXED

Production Issue Discovered & Resolved

During UAT, webhook delays were causing payment status to remain PENDING in the LMS even after the gateway confirmed a successful charge. Users were not getting course access despite being charged — a P0 defect.

- Immediately blocked the release and escalated to the tech team
- Coordinated manual reconciliation of gateway logs against LMS records for affected users
- Worked with developers to implement webhook retry handling
- Re-ran all webhook UAT scenarios before sign-off

Outcomes & Key Learnings



What Worked Well

- End-to-end BA ownership from requirements to sign-off reduced ambiguity and rework across all sprint deliverables.
- API testing in Postman gave the BA chair technical credibility and surfaced contract-level issues before they hit UAT.
- RTM-driven UAT ensured every FRD requirement had a traceable test result and audit trail for sign-off confidence.
- SQL reconciliation post-go-live caught the webhook delay issue before it became a user-reported problem.

What I Would Do Differently

- Involve finance and operations during requirement gathering, not just FRD review — their late inputs caused rework and sprint re-planning.
- Push for performance and latency testing earlier in the sprint cycle rather than discovering API latency issues during UAT.

BA Skills Demonstrated

- Requirements: BRD, FRD, MoSCoW prioritisation
- Documentation: RTM, UAT plan, acceptance criteria
- Technical: REST API testing (Postman), HTTP status codes,
- Data: SQL — funnel analysis, reconciliation, before/after comparison
- Process: Agile sprint ceremonies, backlog grooming, scope management
- Stakeholder: Cross-functional facilitation, conflict resolution, and negotiation

Future Improvements Proposed

- **Intelligent gateway routing** — auto-route by country success rates
- **Real-time ops dashboard** — live visibility into failure spikes and trends
- **Smarter retry logic** — distinguish retrievable vs non-retrievable errors

"As Business Analyst, my role was not just to document requirements — it was to ensure every stakeholder's need was translated into a working, testable, and measurable outcome that the business could confidently sign off."

Dibyashree Chowdhury	Business Analyst Hyderabad, India	Available for BA roles across Fintech, Healthcare IT, SaaS & Ed-Tech
----------------------	-------------------------------------	--